



Angular Avançado

Sessão 7

Fechar o Router · Serviços e Dependency Injection

João Gonçalves · joaogoncalves.net · Making the complicated simple

Onde ficamos na Sessão 6

Router — das primeiras rotas ao loadChildren

Rotas

- path → component
- '' + pathMatch: 'full'
- ** em último
- routerLink · navigate()

Parâmetros

- :id · matrix ;x=1
- ActivatedRoute.param Map
- → refactor para inputs: withComponentInputBinding()

Child routes

- children + outlet filho
- BookingNavigation
- .. sobe **um segmento**

Lazy loading

- ticketing.routes.ts
- loadChildren + import()
- Prefixo /ticketing/...

Hoje: acabar o Router e começar a tirar lógica dos componentes — serviços e Dependency Injection.

Agenda de hoje

01 Arrumar a casa O que mudou no repo desde a Sessão 6 ≈ 5 min

02 Lazy loading (cont.) loadComponent · verificar os chunks · preloading ≈ 15 min

03 Query string e URLs queryParams · fragment · path vs hash ≈ 17 min

04 Serviços e DI FlightClient · inject() · ConfigService ≈ 70 min

Arrumar a casa: já está no repo

git pull origin main — um commit por cima da tag sessao-06

Onde	Estava	Sintoma	Agora
home.ts	navigate(['/booking/flight-search'])	volta a /home (o ** apanha)	'/ticketing/booking/flight-search'
sidebar	isActive('/boarding', ...)	Boarding nunca fica ativo	routerLinkActive no
app.routes.ts	imports de FlightSearch, FlightEdit, ...	nenhum, mas engana	apagados: o que é lazy não se importa
flight-search.html	class="action" no Edit	Edit cai no slot default	class="actions"
FlightCard	computed sem return	<p> vazio no card	devolve 'from → to'

Nenhum dava erro na consola. URLs **absolutos** em strings (navigate, isActive, routerLink="/...") têm de ser revistos sempre que o prefixo muda; os **relativos** ficaram intactos.

Regra prática: relativo dentro de uma área; absoluto só para saltar entre áreas (sidebar, Home). Também saíram imports e código comentado que já não se usavam.



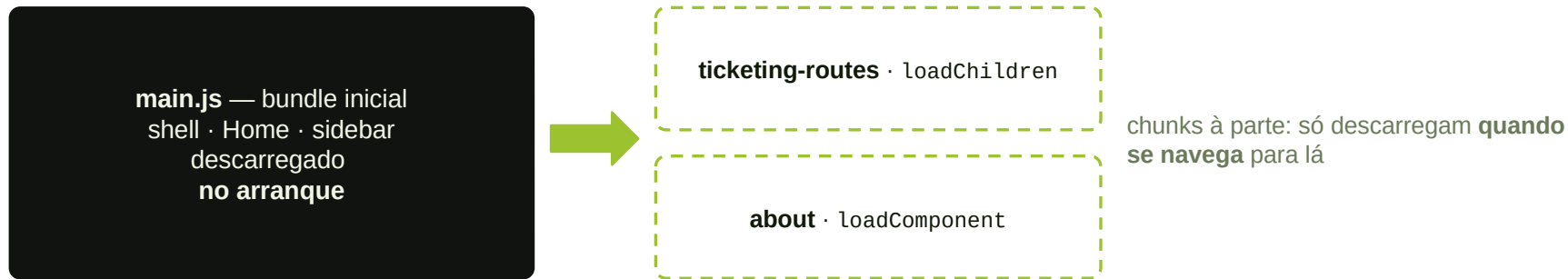
Módulo 3 · Routing

Lazy loading (continuação)

Um componente a pedido, confirmar no browser e pré-carregar em segundo plano

Lazy loading: o que vai no arranque

O `import()` dinâmico parte a app em ficheiros



	Carrega	Quando usar	No Skyboard
loadChildren	um conjunto de rotas (uma área)	features com várias páginas	ticketing (Sessão 6)
loadComponent	um só componente	página isolada, pesada ou rara	About (hoje)

O que é lazy **não se importa** estaticamente em lado nenhum — senão volta para o bundle inicial.

Quando descarregar: três estratégias

Estratégia	Arranque	1. ^a visita a uma área	Custo
Eager (tudo no main)	mais lento	imediate	bundle inicial grande
Lazy · NoPreloading (omissão)	rápido	espera pelo chunk	pequena espera ao navegar
Lazy · PreloadAllModules	rápido	imediate (já descarregou)	descarrega tudo, mesmo o que não se visita

Preloading

- Arranca sem os chunks lazy
- Logo a seguir, descarrega-os em segundo plano
- Se o utilizador for mais rápido: lazy normal

Estratégias custom

- `PreloadingStrategy` própria — rara
- `ngx-quicklink`: quando o link fica visível
- Primeiro, ver se as incluídas chegam



Da teoria ao código

Agora, vamos escrever o código

- O About passa a loadComponent
- Confirmar os chunks: no terminal e na Network
- Ativar o preloading e ver os chunks a descarregar sozinhos

Ficheiros: `about.ts` · `app.routes.ts` · `app.config.ts`

loadComponent e como confirmar que funciona

```
um só componente
// shell/about/about.ts
export default class About {}

// app.routes.ts
{
  path: 'about',
  loadComponent: () => import('./shell/about/about'),
},
```

```
ng build (development) – Skyboard
```

Initial chunk files	Names	Raw size
main.js	main	9.01 kB
...		
Lazy chunk files	Names	Raw size
chunk-....js	ticketing-routes	703.86 kB
chunk-....js	about	1.48 kB

No browser

DevTools → Network → filtro JS. Arrancar em /home: os chunks lazy só descarregam ao clicar em Booking ou About. Em dev aparecem muitos ficheiros extra (compilação a pedido); em produção são poucos.

Preloading: lazy no arranque, pronto quando for preciso

```
app.config.ts
import {
  PreloadAllModules, provideRouter, withComponentInputBinding,
  withPreloading,
} from '@angular/router';

provideRouter(
  routes,
  withComponentInputBinding(),
  withPreloading(PreloadAllModules),
),
```

Resumo

- Uma linha no provideRouter
- PreloadAllModules: tudo, logo a seguir ao arranque
- Omissão: NoPreloading

Na Network: arrancar em /home e ver ticketing-routes e about a descarregar sozinhos, logo depois da página aparecer.

Arranque tão rápido como sem nada lazy · navegação tão rápida como com tudo carregado.



Módulo 3 · Routing

Query string e URLs

Parâmetros para o URL inteiro — e como o URL chega ao servidor

As partes de um URL

Quatro partes, quatro donos diferentes

<code>/ticketing/booking/flight-edit/3</code>	<code>;showDetails=true</code>	<code>?expertMode=true</code>	<code>#details</code>
path + segmento	matrix	query	fragment

Parte	Pertence a	Chega ao componente como	Uso típico
Segmento :id	uma rota	<code>input · paramMap</code>	identificar o recurso
Matrix ;x=1	um segmento	<code>input · paramMap</code>	opções dessa rota
Query ?x=1	o URL inteiro	<code>input · queryParamMap</code>	definições transversais, filtros partilháveis
Fragment #x	o URL inteiro (uma string)	<code>ActivatedRoute.fragment</code>	âncora, secção da página

Com `withComponentInputBinding()`, tudo chega a inputs — **menos o fragment**, que não é chave-valor.

Path vs hash: o que chega ao servidor

PathLocationStrategy (omissão)

- URL: /ticketing/booking
- Refresh → o browser pede GET /ticketing/booking
- O servidor tem de devolver o index.html (rewrite)
- <base href="/">: onde começa a app

HashLocationStrategy

- URL: /#/ticketing/booking
- Refresh → o browser pede GET /: o #... nunca sai do browser
- Nada a configurar no servidor
- Em troca: URLs menos naturais, sem SSR por rota

Regra: **path routing**, com o servidor configurado. Hash só quando não se controla o servidor.



Da teoria ao código

Agora, vamos escrever o código

- Um input `expertMode` no `FlightEdit`, lido da query string
- O link com `queryParams`, `merge` e `fragment` — e o mesmo com `navigate()`
- Experimentar `withHashLocation()` (temporário) e voltar ao `path`

Ficheiros: `flight-edit.ts` · `flight-edit.html` · `app.config.ts`

Query string e hash fragment

```
FlightEdit — ?expertMode=true#details
// flight-edit.ts
readonly expertMode = input(false, { transform: booleanAttribute });

<!-- flight-edit.html · [] = a rota atual -->
<a [routerLink]="[]"
  [queryParams]="{ expertMode: true }"
  queryParamsHandling="merge"
  fragment="details">Expert mode</a>

// o mesmo, a partir do código
this.router.navigate([], {
  queryParams: { expertMode: true },
  queryParamsHandling: 'merge',
  fragment: 'details',
});
```

Regras

- merge junta à query atual; preserve mantém-na tal como está
- Query params → inputs (com o input binding)
- Fragment **não** vira input → `ActivatedRoute.fragment`
- Mesmo componente, só o input muda: **não** é recriado

Resultado: `/ticketing/booking/flight-edit/3;showDetails=true?expertMode=true#details`

Experimentar: hash routing e base href

```
hash routing – mostrar e remover
// app.config.ts – TEMPORÁRIO
provideRouter(
  routes,
  withComponentInputBinding(),
  withPreloading(PreloadAllModules),
  withHashLocation(),           // → localhost:4200/#/home
),
```

Verificar

- A raiz abre em `/#/home`
- Os links passam a ter `#`
- **Remover** antes do commit

```
# publicar numa subpasta (só afeta o dist)
ng build --base-href /skyboard/
```

```
# servidor (ex.: nginx) – path routing em produção
location / { try_files $uri $uri/ /index.html; }
```

Sem o rewrite, um refresh em `/ticketing/booking` dá **404** em produção. O `ng serve` já o faz por nós.

Router: o capítulo completo

Rotas e links

- Routes: path → component
- '+' + pathMatch: 'full' · ** em último
- routerLink · routerLinkActive · isActive()
- router.navigate(..., extras)

Parâmetros e hierarquia

- Segmento :id · matrix ;x=1 · query ?x=1
- withComponentInputBinding() → inputs
- children + outlet filho
- .. sobe um **segmento**

Carregamento e URL

- loadChildren · loadComponent
- withPreloading(PreloadAllModules)
- Path (omissão) vs withRouterLocation()
- Sair da rota **destrói** o componente

Falta responder a uma pergunta da Sessão 6: porque é que a pesquisa recomeça do zero quando voltamos a Flights?



A seguir

Serviços e Dependency Injection

Estado e lógica que sobrevivem à navegação — capítulo 5

Dois problemas no FlightSearch

A pergunta que ficou da Sessão 6 — e outra que ainda não fizemos

1 · O estado não sobrevive

- Pesquisar Faro → Porto, ir a About, voltar
- A pesquisa e o cesto voltam ao início
- Sair da rota **destrói** o componente — e os signals dele

2 · O componente faz tudo

- Sabe o **URL** da API e faz o **pedido HTTP**
- Guarda o **estado** (filtro, cesto)
- E ainda trata da **UI**

Plano: acesso a dados num **serviço** → como o Angular entrega serviços (**DI**) → estado num serviço que não morre com o componente (**store**).



Bloco A · 5.1.1 – 5.1.5

Serviços e Dependency Injection

O componente pede — o Angular decide o que entrega

■ ■ ■ O que é uma dependência?

■ ■ ■ Definição

Tudo aquilo de que um pedaço de código **precisa** para fazer o seu trabalho — e que **não é ele** que implementa.

FlightSearch



FlightClient



URL

mostra voos → precisa de quem os vá buscar → que precisa de saber onde

a dependência escondida

```
class FlightSearch {  
  search(from: string, to: string) {  
    const client = new FlightClient(  
      'https://api.skyboard.dev',  
    );  
    return client.find(from, to);  
  }  
}
```

```
// a assinatura não diz que isto faz HTTP,  
// nem para onde
```

O problema não é ter dependências — é **escondê-las** e **fixá-las** lá dentro.

Três maneiras de obter uma dependência

	No código	Quem escolhe a implementação?	Para testar sem HTTP...
1 • Criar	<code>new FlightClient(url)</code>	o consumidor	mexer no consumidor
2 • Ir buscar	<code>FlightClient.getInstance()</code> <code>registry.get('flights')</code>	o consumidor sabe onde procurar	mexer no estado global
3 • Receber	<code>constructor(client: FlightClient)</code> <code>inject(FlightClient)</code>	quem monta a aplicação	trocar a configuração

Só na terceira a decisão **sai** do consumidor. É isto que se chama **Dependency Injection**.

`inject()` parece "ir buscar", mas o consumidor só **nomeia** o que quer. O que recebe decide-o a configuração.

Inversion of Control

Quem chama quem?

Controlo nosso — biblioteca

- O **nosso** código chama, quando quer
- `Math.max(...)`, `date-fns`, `http.get(...)`
- Nós decidimos o quê, quando e com quê

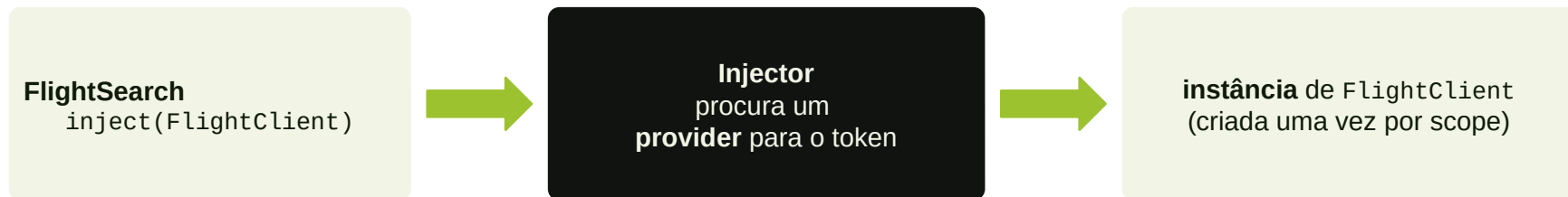
Controlo invertido — framework

- A **framework** chama o nosso código
- `Constructor`, `ngOnDestroy`, `guards`, o componente de uma rota
- "Não nos ligue, nós ligamos-lhe"

Já o conhecem: nunca fizeram `new FlightSearch()`. O Router decide quando o criar; o `@if` decide quando um bloco nasce e morre.

DI é IoC aplicada às dependências: quem cria e liga os objetos é o Angular, não o nosso código.

Os papéis: token, provider, injector, instância



Termo	O que é	No café	No exemplo
Token	o que se pede ao inject	"um café"	FlightClient
Provider	a receita: o que entregar para esse token	a receita da casa	@Service() · providers: [...]
Injector	conhece as receitas e guarda as instâncias	o balcão	root · rota · componente
Instância	o objeto entregue, partilhado no scope	a chávena	um FlightClient para a app

Já o fizemos sem lhe dar nome: `inject(Router)`, `inject(HttpClient)`, `inject(ActivatedRoute)`.

Porquê o trabalho todo?

Baixo acoplamento

- O FlightSearch sabe **o quê** (voos)
- Não sabe **como** nem **de onde**

Testabilidade

- No teste: um FlightClient **falso** com 3 voos fixos
- Sem HTTP, sem API, sem esperar

Reutilização

- O mesmo componente noutro contexto
- Outra API, demo, outro cliente — **outra configuração**

O custo: **indireção**. "De onde vem este objeto?" deixa de estar à vista → Angular DevTools, separador **Injector Tree**.

Porque é que o inject() precisa de contexto?

o Angular está a **criar** o FlightSearch
"injector atual" = o do componente

mais tarde: um **clique** chama search()
"injector atual" = nenhum

tempo

Dentro da janela ✓

- Field initializers e constructor
- Factories de providers

Fora da janela ✗

- Métodos chamados por eventos, setTimeout, subscribe
- inject() não sabe a quem perguntar → NG0203

Como o this em JavaScript: não depende de **onde** o código está escrito, mas de **quando** corre.



Da teoria ao código

Agora, vamos escrever o código

- Gerar o FlightClient e mover o acesso a dados para lá
- Injetá-lo no FlightSearch com inject()
- Onde o inject() funciona — e onde dá NG0203
- Um serviço que pede outro serviço

Ficheiros: `flight-client.ts` · `flight-search.ts` · `config-service.ts`

O primeiro serviço: FlightClient

Acesso a dados de voos, fora do componente

```
ng g s domains/ticketing/data/flight-client
```

data/flight-client.ts

```
// Angular 22+  
import { Service } from '@angular/core';
```

```
@Service()  
export class FlightClient {}
```

o que vão encontrar em código existente

```
// antes do Angular 22 – mesma semântica  
import { Injectable } from '@angular/core';
```

```
@Injectable({ providedIn: 'root' })  
export class FlightClient {}
```

Singleton na app

@Service() registra a classe no **root injector**: uma instância para a aplicação inteira, compartilhada por todos os que a pedem. O CLI já não põe o sufixo Service — para acesso a dados usamos Client, como o HttpClient.

Mover o acesso a dados para o serviço

```
data/flight-client.ts
import { httpResource } from '@angular/common/http';
import { Service, Signal } from '@angular/core';
import { API_URL } from './api';
import { Flight } from './flight';

@Service()
export class FlightClient {
  findResource(from: Signal<string>, to: Signal<string>) {
    return httpResource<Flight[]>(
      () => {
        if (!from() || !to()) {
          return undefined; // sem pedido
        }
        return { url: API_URL, params: { from: from(), to: to() } };
      },
      { defaultValue: [] },
    );
  }
}
```

O que mudou

- O filtro chega como `Signal<string>` — o serviço não sabe de onde vem
- O URL e o `httpResource` saem do componente
- O `httpResource` faz `inject(HttpClient)` por dentro
→ atenção ao **injection context**

Usar o serviço no FlightSearch

```
feature-booking/flight-search/flight-search.ts
export class FlightSearch {
  private readonly flightClient = inject(FlightClient);

  protected readonly filter = signal({ from: 'Lisbon', to: 'Porto' });
  protected readonly filterForm = form(this.filter);

  // ANTES do resource: os field initializers correm por ordem
  protected readonly from = computed(() => this.filter().from);
  protected readonly to = computed(() => this.filter().to);

  protected readonly flightsResource =
    this.flightClient.findResource(this.from, this.to);

  protected readonly flights = this.flightsResource.value;
  // error · isLoading · status · basket · ... iguais
}
```

Resultado

- Saem do componente:
HttpClient, httpResource,
API_URL
- O template **não muda**
- O componente já não sabe **como**
se obtêm os voos

Injection context: onde o inject() funciona

onde funciona — e onde não

```
// + Injector, runInInjectionContext no import de '@angular/core'
export class FlightSearch {
  private readonly client = inject(FlightClient); // ✅ field initializer
  private readonly injector = inject(Injector);

  constructor() { inject(Router); } // ✅ constructor

  protected search(): void {
    inject(FlightClient); // ❌ NG0203
    this.client.findResource(...); // ❌ idem (httpResource)

    runInInjectionContext(this.injector, () => {
      inject(FlightClient); // ✅ mas raro
    });
  }
}
```

Regras

- inject em **field initializers** e no **constructor**
- Funções que usam inject por dentro (httpResource, findResource) herdam a regra
- assertInInjectionContext(fn) falha cedo, com o nome da função

Por isso o resource é criado no field initializer — e o search() só faz reload().

❏ Serviços que pedem serviços

FlightSearch



FlightClient



ConfigService

```
// domains/shared/util-common/config-service.ts
@Service()
export class ConfigService {
  readonly apiUrl = 'http://localhost:3000/flights';
}
```

```
// data/flight-client.ts
@Service()
export class FlightClient {
  private readonly config = inject(ConfigService);

  findResource(from: Signal<string>, to: Signal<string>) {
    // ... url: this.config.apiUrl
  }
}
```

O Angular resolve a cadeia sozinho. Ninguém faz new — e cada peça pode ser trocada sem mexer nas outras.

Onde ficámos

O capítulo 5 continua na Sessão 8

Hoje: serviços e DI

- **DI** = IoC aplicada às dependências
- Token · provider · injector · instância
- `@Service()` → `FlightClient` no root
- `inject()` só num **injection context**
- `FlightSearch` → `FlightClient` → `ConfigService`

Sessão 8

- **Providers**: trocar implementações sem mexer em quem pede
- **Scopes**: root, rota e componente
- **Stores**: estado que sobrevive à navegação

Problema 2 resolvido: o componente já não sabe de HTTP nem de URLs. O problema 1 — o estado que se perde ao navegar — fica para a Sessão 8.